

Text Editor Project - Phase 1: Enhanced Text Operations

Overview

Building upon your basic text editor from Phase 0, you'll now implement additional text manipulation features and improved cursor movement, making your editor more practical and powerful.

Prerequisites

- Completed Phase 0 implementation with:
 - Basic cursor movement using arrow keys
 - Insert and Normal modes
 - Basic character insertion
 - Working display function

New Features to Implement

1. Delete Operations

Requirements

- **Normal Mode Commands**
 - 'dd': Delete current line
 - 'D': Delete from cursor to end of line
 - 'x': Delete character under cursor
- **Insert Mode**
 - Backspace: Delete character before cursor

Implementation Details

```
class TextEditor {
public:
    void deleteCurrentLine();           // for 'dd' command
    void deleteToEndOfLine();          // for 'D' command
    void deleteCharacterAtCursor();     // for 'x' command
    void backspace();                  // for backspace in insert mode
};
```

Edge Cases to Handle

- Deleting the last line in the document
- Deleting an empty line
- Backspace at the start of a line (should join with previous line)
- Cursor position after deletion

2. Copy and Paste Operations

Requirements

- 'yy': Yank (copy) current line
- 'p': Paste copied content after current line
- 'P': Paste copied content before current line

Implementation Details

```
class TextEditor {
private:
    std::string copyBuffer; // Store copied content

public:
    void yankLine();          // for 'yy' command
    void pasteAfter();        // for 'p' command
    void pasteBefore();       // for 'P' command
};
```

Edge Cases

- Copying an empty line
- Pasting when nothing has been copied
- Pasting at the start or end of the document

3. Advanced Cursor Movement

Requirements

- '0': Move to first character of current line
- '\$': Move to last character of current line
- 'w': Move to start of next word
- 'b': Move to start of previous word
- 'e': Move to end of current/next word

Implementation Details

```
class TextEditor {
private:
    bool isWordCharacter(char c); // Helper for word movement
    bool isPunctuation(char c);   // Helper for word movement

public:
    void moveToStartOfLine();      // for '0' command
    void moveToEndOfLine();        // for '$' command
    void moveToNextWord();         // for 'w' command
};
```

```
void moveToPreviousWord();    // for 'b' command
void moveToWordEnd();        // for 'e' command
};
```

Word Movement Rules

- Words are separated by:
 - Spaces
 - Punctuation marks
 - Line breaks
- Multiple consecutive spaces count as one separator
- Handle transitions between lines properly

4. Display Enhancement

Requirements

- Show line numbers
- Status line with:
 - Current mode (INSERT/NORMAL)
 - Cursor position (Line X, Column Y)
 - Total number of lines
 - Last command executed (optional)

Implementation Details

```
struct EditorStatus {
    Mode currentMode;        // INSERT or NORMAL
    size_t cursorLine;       // Current line number
    size_t cursorColumn;     // Current column number
    size_t totalLines;       // Total lines in document
    std::string lastCommand; // Last executed command
};

class TextEditor {
private:
    EditorStatus status;

public:
    void updateDisplay();
    void updateStatusLine();
    std::string getStatusLineText() const;
};
```

Display Format

```
1 |First line of text
2 |Second line of text
3 |Current line      |  <- cursor
4 |Last line
-----
NORMAL MODE | Line: 3, Col: 5 | Total Lines: 4
```

Error Handling

Your implementation should handle these cases:

1. Commands at document boundaries
2. Operations on empty lines
3. Word movement with special characters
4. Copy/paste with empty buffer

Testing Requirements

Create test cases for:

1. All new commands
2. Edge cases for each operation
3. Combinations of commands
4. Boundary conditions

Example test scenarios:

```
void testDeleteOperations() {
    // Test deleting last line
    // Test deleting empty line
    // Test backspace at line start
}

void testWordMovement() {
    // Test movement with punctuation
    // Test movement across empty lines
    // Test movement at document boundaries
}
```

Evaluation Criteria

1. Correct implementation of all commands
2. Proper handling of edge cases
3. Code organization and clarity
4. Command responsiveness
5. Display formatting and accuracy

Bonus Features (Optional)

1. Number prefixes for commands (e.g., '2dd' to delete 2 lines)
2. Improved word movement handling
3. More informative status line
4. Command history in status line

Submission Requirements

1. Complete source code with all new features
2. README file explaining:
 - How to use each command
 - Any known limitations
 - Bonus features implemented (if any)
3. Test cases demonstrating functionality
4. Brief report on challenges faced and solutions implemented

Implementation Tips

1. Start with simpler commands (like 'x' and 'dd')
2. Implement one feature category at a time
3. Test each feature thoroughly before moving to the next
4. Keep your display function updated as you add features
5. Maintain clean code organization

Remember: Focus on making each feature work correctly before moving on to the next one. Clean, working code is better than partially implemented features.